



Hochiminh City University of Technology
Computer Science and Engineering
[CO1027] - Fundamentals of C++ Programming

Polymorphism

Lecturer: Duc Dung Nguyen
Credits: 3

Outcomes

- ❖ Understand the concept of polymorphism in OOP.

Outline

- ❖ Introduction
- ❖ Type Fields and `switch` Statements
- ❖ Virtual Functions
- ❖ Abstract Base Classes and Concrete Classes
- ❖ Polymorphism
- ❖ New Classes and Dynamic Binding
- ❖ Virtual Destructors

Introduction

- **virtual** functions and polymorphism
 - Design and implement systems that are more easily extensible
 - Programs written to generically process objects of all existing classes in a hierarchy

Type Fields and `switch` Statements

- **`switch`** statement
 - Take an action on a object based on its type
 - A **`switch`** structure could determine which **`print`** function to call based on which type in a hierarchy of **`shapes`**
- Problems with **`switch`**
 - Programmer may forget to test all possible cases in a switch.
 - Tracking this down can be time consuming and error prone
 - `virtual`** functions and polymorphic programming can eliminate the need for **`switch`**

Virtual Functions

- **virtual** functions

- Used instead of **switch** statements

- Declaration:

- Keyword **virtual** before function prototype in base class

- virtual void draw() const;**

- A base-class pointer to a derived class object will call the correct **draw** function

- If a derived class does not define a **virtual** function it is inherited from the base class.

Virtual Functions (cont)

- **ShapePtr->Draw()** ;
 - Compiler implements dynamic binding (late binding)
 - Function determined during execution time
- **ShapeObject.Draw()** ;
 - Compiler implements static binding (early binding)
 - Function determined during compile-time

Abstract and Concrete Classes

- Abstract classes
 - Sole purpose is to provide a base class for other classes
 - No objects of an abstract base class can be instantiated
 - Too generic to define real objects, i.e. **TwoDimensionalShape**
 - Can have pointers and references
 - Concrete classes - classes that can instantiate objects
 - Provide specifics to make real objects , i.e. **Square, Circle**

Abstract and Concrete Classes (cont)

- Making abstract classes
 - Declare one or more **virtual** functions as “pure” by initializing the function to zero

virtual double earnings() const = 0;

- Pure **virtual** function

Polymorphism

- Polymorphism:
 - Ability for objects of different classes to respond differently to the same function call
 - Base-class pointer (or reference) calls a **virtual** function
 - C++ chooses the correct overridden function in object
 - Suppose **print** not a **virtual** function

```
Employee e, *ePtr = &e;  
HourlyWorker h, *hPtr = &h;  
ePtr->print();    //call base-class print function  
hPtr->print();    //call derived-class print function  
ePtr=&h;          //allowable implicit conversion  
ePtr->print();    // still calls base-class print
```

New Classes and Dynamic Binding

- Dynamic binding (late binding)
 - Object's type not needed when compiling **virtual** functions
 - Accommodate new classes that have been added after compilation
 - Important for ISV's (Independent Software Vendors) who do not wish to reveal source code to their customers

Virtual Destructors

- Problem:
 - If base-class pointer to a derived object is **deleted**, the base-class destructor will act on the object
- Solution:
 - Declare a **virtual** base-class destructor
 - Now, the appropriate destructor will be called